

Computer Science & IT

C programming



Function & Storage Class

Lecture No. 01



By- Abhishek Sir

Recap of Previous Lecture



Topic

break

Topic

continue

Topic

Topic

Topic

Topics to be Covered



Topic

function

Topic

Activation Record

Topic

Activation tree → [function management]
↓
Recursion

Topic

Global variable

Topic



Function



- *What is a function:* A function is a self-contained block of code designed to perform a specific task. It allows you to organize your program into smaller, manageable, and reusable units.

block of code that perform specific task called function

Size of program increases then its important to management the program



Function



```
#include <stdio.h>
```

```
int fun(int, int);
```

```
int main(){
```

```
    int a = 10, b = 20;
```

```
    printf("%d", fun(a,b));
```

```
}
```

```
int fun(int x,int y){
```

```
    int z;
```

```
    z = x++* ++y;
```

```
    return z;
```

```
}
```



forward declaration by giving prototype of function

return type Name of function (parameter list)



function call fun(a,b) - a,b, argument actual parameters
control transfer to definition of function



Function



```
#include <stdio.h>
```

```
int fun(int, int);
```

```
int main(){
```

```
    int a = 10, b = 20;
```

```
    printf("%d", fun(a,b));
```

```
}
```

```
int fun(int x, int y){
```

```
    int z;
```

```
    z = x++ * ++y;
```

```
    return z;
```

```
}
```

- 1 forward declaration
- 2 call the function
definition of function. x, y parameter or formal parameter
- 3 control transfer and actual parameters copied to formal parameters
 $x=a, y=b$ Call by value
- 4 function execution $z = 10 * 21 = 210$
and return the value if present

void fun() ← function taking

no parameter and no return value

void fun(int, int)

int fun()



Function



```
#include <stdio.h>
```

```
int fun(int x,int y){
```

```
    int z;
```

```
    z = x++* ++y;
```

```
    return z;
```

```
}
```

```
int main(){
```

```
    int a = 10, b = 20;
```

```
    printf("%d", fun(a,b));
```

```
}
```

* variable declared with { } (block)

is Local variable

x, y, z Local variable

* when function called ^{Local} variables are created

* upon termination of function Local variable destroyed.



Function



```
#include <stdio.h>
```

```
int fun(int, int);
```

```
int c = 12;
```

```
int main(){
```

```
    int a = 10, b = 20;
```

```
    c++;
```

```
    printf("%d", fun(a,b));
```

```
}
```

13 → global variable

f + c ✓

243

x = 10, y = 20

```
int fun(int x, int y){
```

```
    int z;
```

```
    c = c + 20;
```

```
    z = x++ * ++y + c;
```

```
    return z;
```

```
}
```

z = 10 * 21 + 33

210 + 33 = 243

expression
↑
20000

Scope:

where we can access

(read/write) the variable

global variable

scope is through the program



Function



```
#include <stdio.h>
```

```
int g = 5 8
```

```
int foo(int x, int y) {
```

```
    int z;
```

```
    z = ++x * y--;
```

```
    g += x;
```

```
    return z;
```

```
}
```

```
int bar(int a) {
```

```
    return a * g--;
```

```
}
```

$3 * 9 = 27$

```
int main() {
```

```
    int a = 3, b = 4;
```

```
    int r1 = foo(a, b);
```

```
    int r2 = bar(a);
```

```
    printf("%d %d", r1, r2);
```

```
}
```

(A) 16 27 ✓

(B) 12 15

(C) 16 24

(D) 20 27

[A]

16, 27

$a = 3, b = 4$

$r_1 = \text{foo}(3, 4)$

$r_1 = 16$

$r_2 = \text{bar}(3)$

$= 27$



Activation Tree and Records



Activation Record: When a function is in execution the

compiler maintains a set of record related with

Activation (function is in execution) call Activation Record

Local variable, return value, parameter passed ect



Activation Tree and Records

When a function is called Activation Record is created
and pushed in runtime stack and upon termination
popped from stack



Activation Tree and Records



Example: Consider the following program

```
int main(){  
    int a,b;
```

```
    .  
    .  
    .  
    f10; →  
    f20;  
}
```

```
void f10(){  
    int b,c;
```

```
    .  
    .  
    .  
    f20;  
}
```

```
Void f20(){  
    int c,d;
```

```
    .  
    .  
    .  
    }  
}
```

Stack

Main
Int a, b



Activation Tree and Records

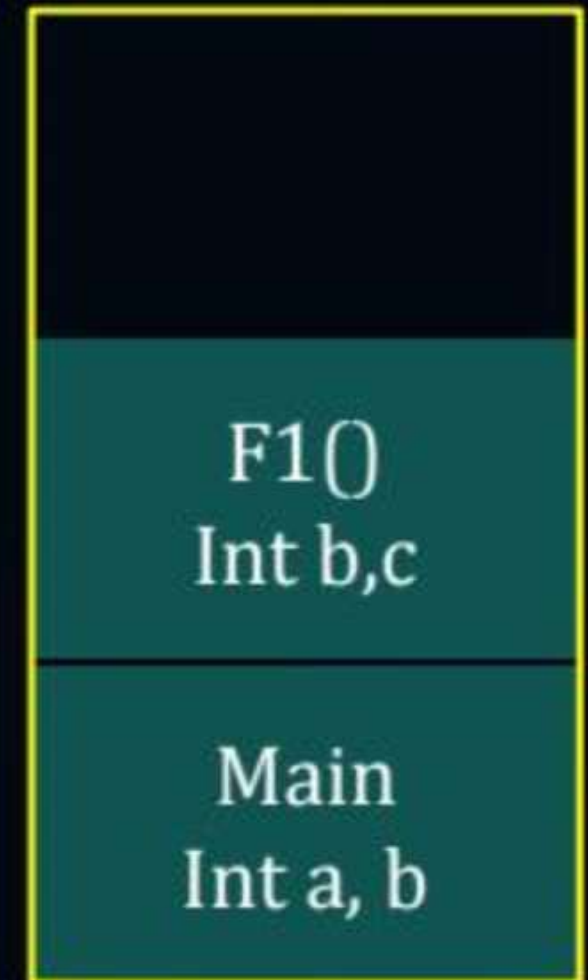


```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f10;  
    f20;  
}  
  
void f10{  
    int b,c;  
    .  
    .  
    .  
    f20;  
}  
  
Void f20{  
    int c,d;  
    .  
    .  
    .  
}
```

Diagram illustrating the activation tree and records:

- The `main` function calls `f10`.
- The `f10` function calls `f20`.

Stack





Activation Tree and Records



```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f10;  
    f20;  
}
```

```
void f10{  
    int b,c;  
    .  
    .  
    .  
    f20;  
}
```

```
Void f20{  
    int c,d;  
    .  
    .  
    .  
    }  
    ↖
```

Stack

F20 Int c , d
F10 Int b,c
Main Int a, b



Activation Tree and Records



```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f10;  
    f20;  
}
```

```
void f10{  
    int b,c;  
    .  
    .  
    .  
    f20;  
}
```

```
Void f20{  
    int c,d;  
    .  
    .  
    .  
}
```

Stack

F20 Int c , d
F10 Int b,c
Main Int a, b



Activation Tree and Records

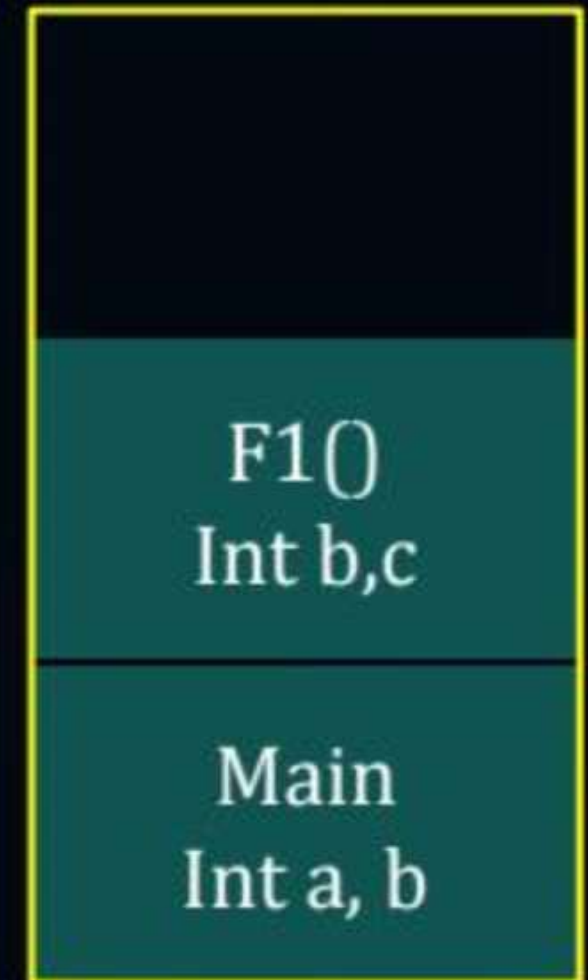


```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f10;  
    f20;  
}
```

```
void f10{  
    int b,c;  
    .  
    .  
    .  
    f20;  
}
```

```
Void f20{  
    int c,d;  
    .  
    .  
    .  
}
```

Stack





Activation Tree and Records



```
int main(){  
    int a,b;
```

```
    .  
    .  
    .
```

```
    f10;  
    f20;
```

```
}
```

```
void f10(){  
    int b,c;
```

```
    .  
    .  
    .
```

```
    f20;
```

```
Void f20(){  
    int c,d;
```

```
    .  
    .  
    .
```

```
}
```

Stack

Main
Int a, b



Activation Tree and Records



```
int main(){  
  int a,b;  
  .  
  .  
  .  
  f10;  
  f20;  
}
```

```
void f10{  
  int b,c;  
  .  
  .  
  .  
  f20;  
}
```

```
Void f20{  
  int c,d;  
  .  
  .  
  .  
}
```

Stack





Activation Tree and Records



```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f10;  
    f20;  
}
```

```
void f10{  
    int b,c;  
    .  
    .  
    .  
    f20;  
}
```

```
Void f20{  
    int c,d;  
    .  
    .  
    .  
}
```

Stack

Main
Int a, b



Activation Tree and Records

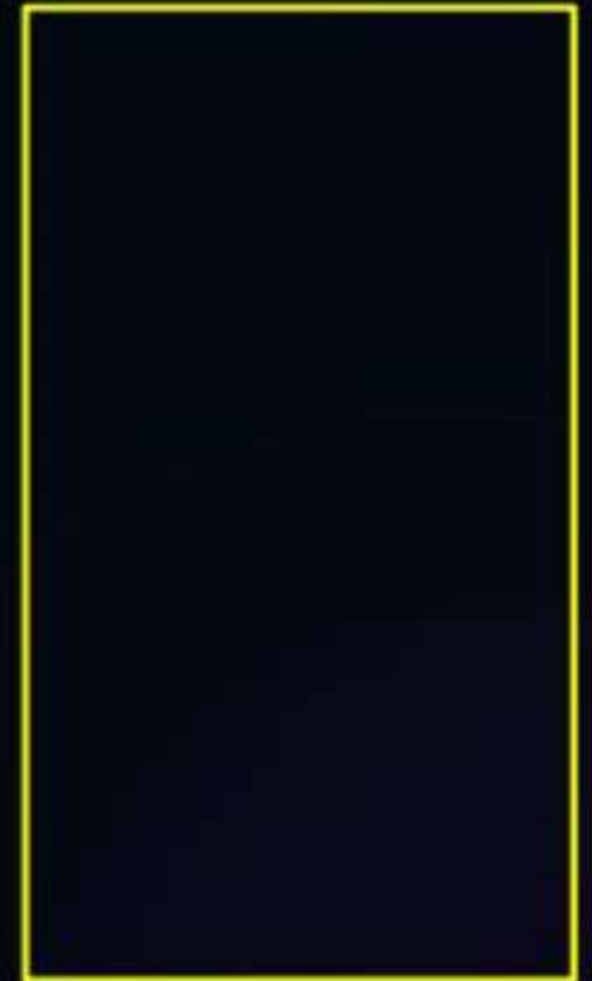


Stack

```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f10;  
    f20;  
}
```

```
void f10{  
    int b,c;  
    .  
    .  
    .  
    f20;  
}
```

```
Void f20{  
    int c,d;  
    .  
    .  
    .  
}
```





Activation Tree and Records

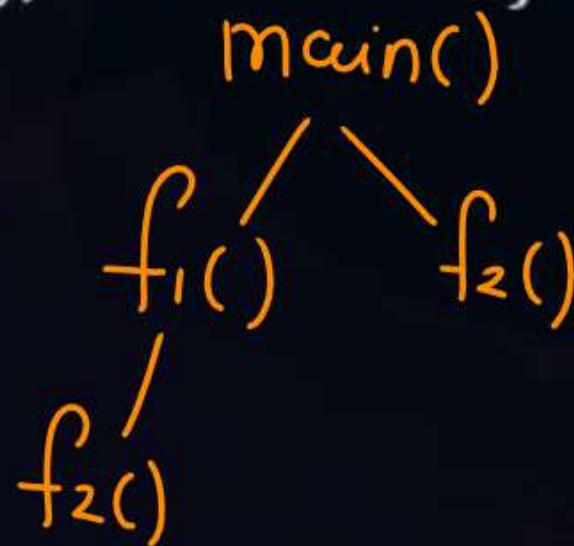


```
int main(){  
    int a,b;  
    .  
    .  
    .  
    f1();  
    f2();  
}
```

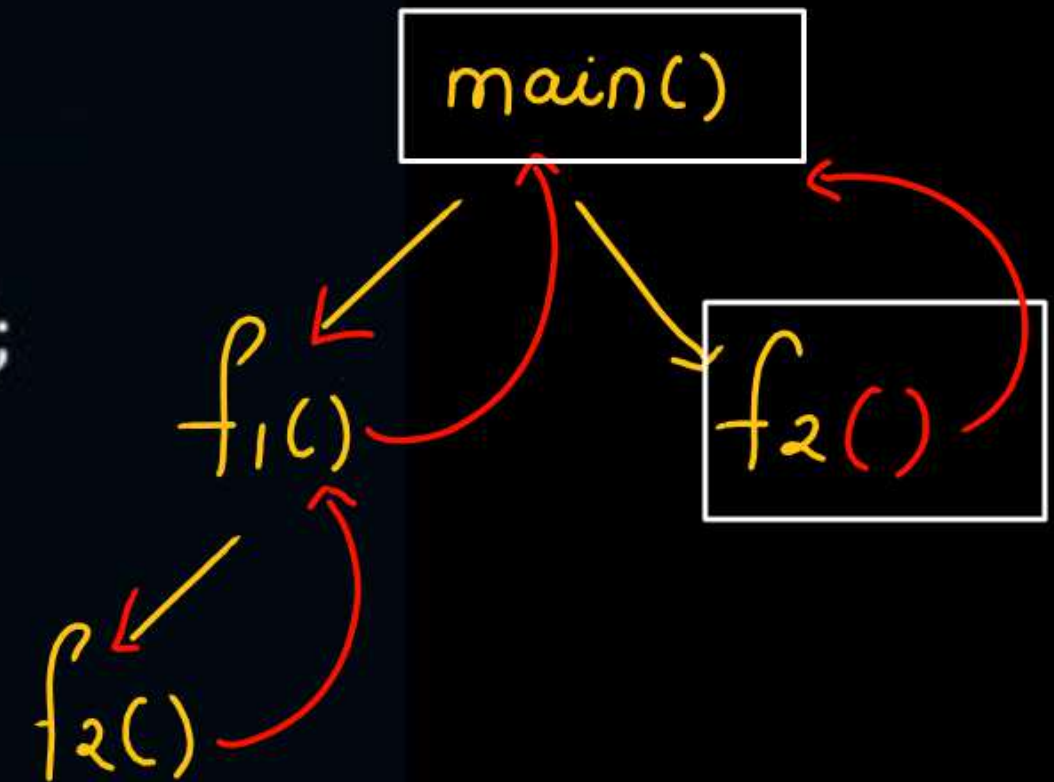
```
void f1(){  
    int b,c;  
    .  
    .  
    .  
    f2();  
}
```

```
Void f2(){  
    int c,d;  
    .  
    .  
    .  
}
```

Activation tree



Tree





Question



#Q

No. of times '*' will be printed by the following C code is _____

```
#include<stdio.h>
```

```
void foo(int x)
```

```
{
```

```
    switch(x){
```

```
        case 1: printf("*");
```

```
        case 2: printf("*");
```

```
        case 3: printf("*");
```

```
        default: printf("*");
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    foo(2.5);
```

```
}
```

X = 2.5

x 2

Switch(2)



Number of star printed is ?

(A) 2

☒ (B) 3

(C) 4

(D) 5



Question



#Q The number of character printed by the code _____

```
#include<stdio.h>
```

```
void a();
```

```
void b();
```

```
void c();
```

```
int main() {
```

```
    a();
```

```
    b();
```

```
    return 0;
```

```
}
```

```
void a() { printf("a"); b(); }
```

```
void b() { printf("a"); c(); }
```

```
void c() { printf("a"); }
```

(A) 2

(B) 3

(C) 4

(D) 5



Question



#Q The number of character printed by the code _____

```
#include<stdio.h>
```

```
void a();
```

```
void b();
```

```
void c();
```

```
int main() {
```

```
    a();
```

```
    b();
```

```
    return 0;
```

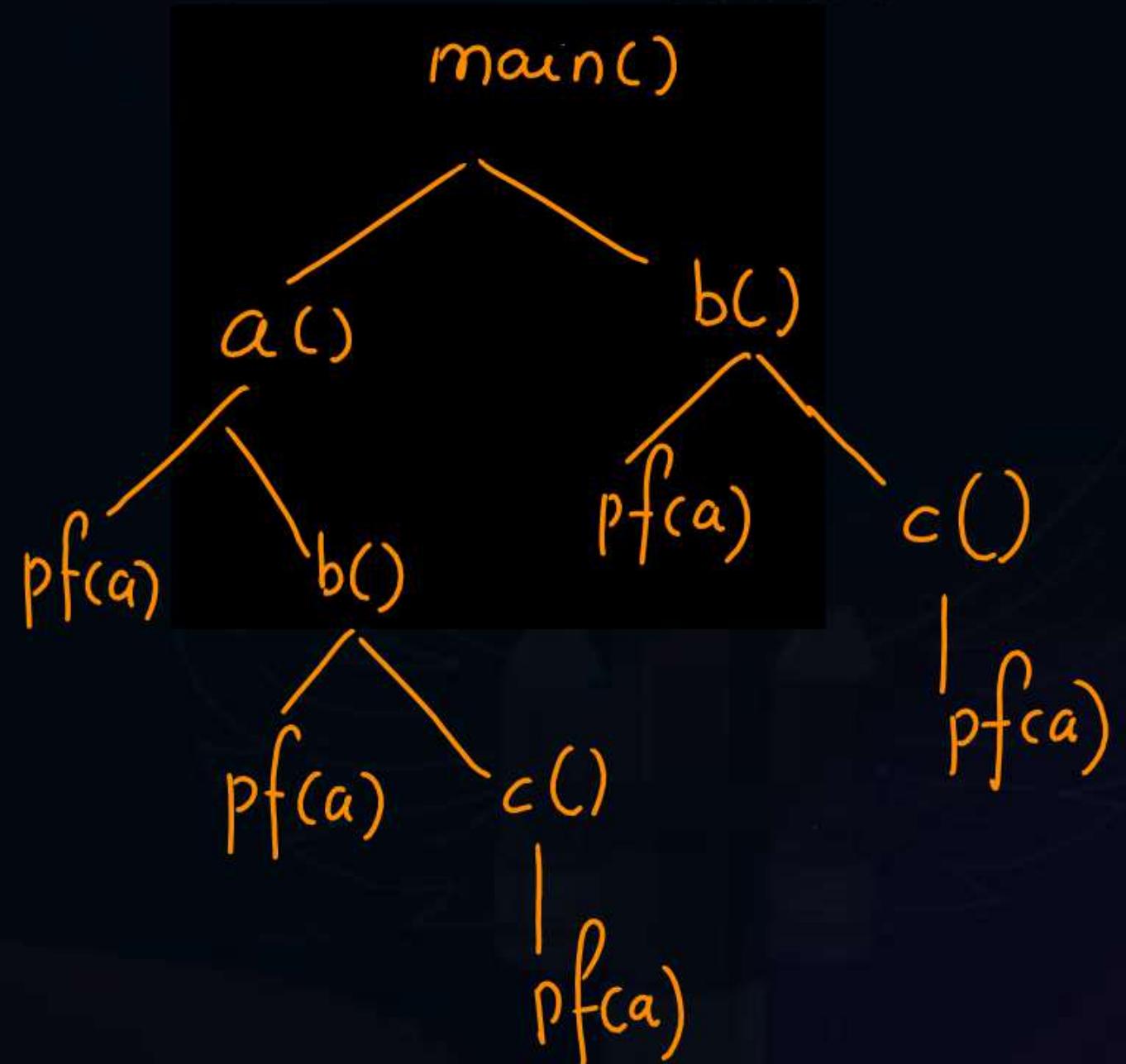
```
}
```

```
void a() { printf("a"); b(); }
```

```
void b() { printf("a"); c(); }
```

```
void c() { printf("a"); }
```

5 times





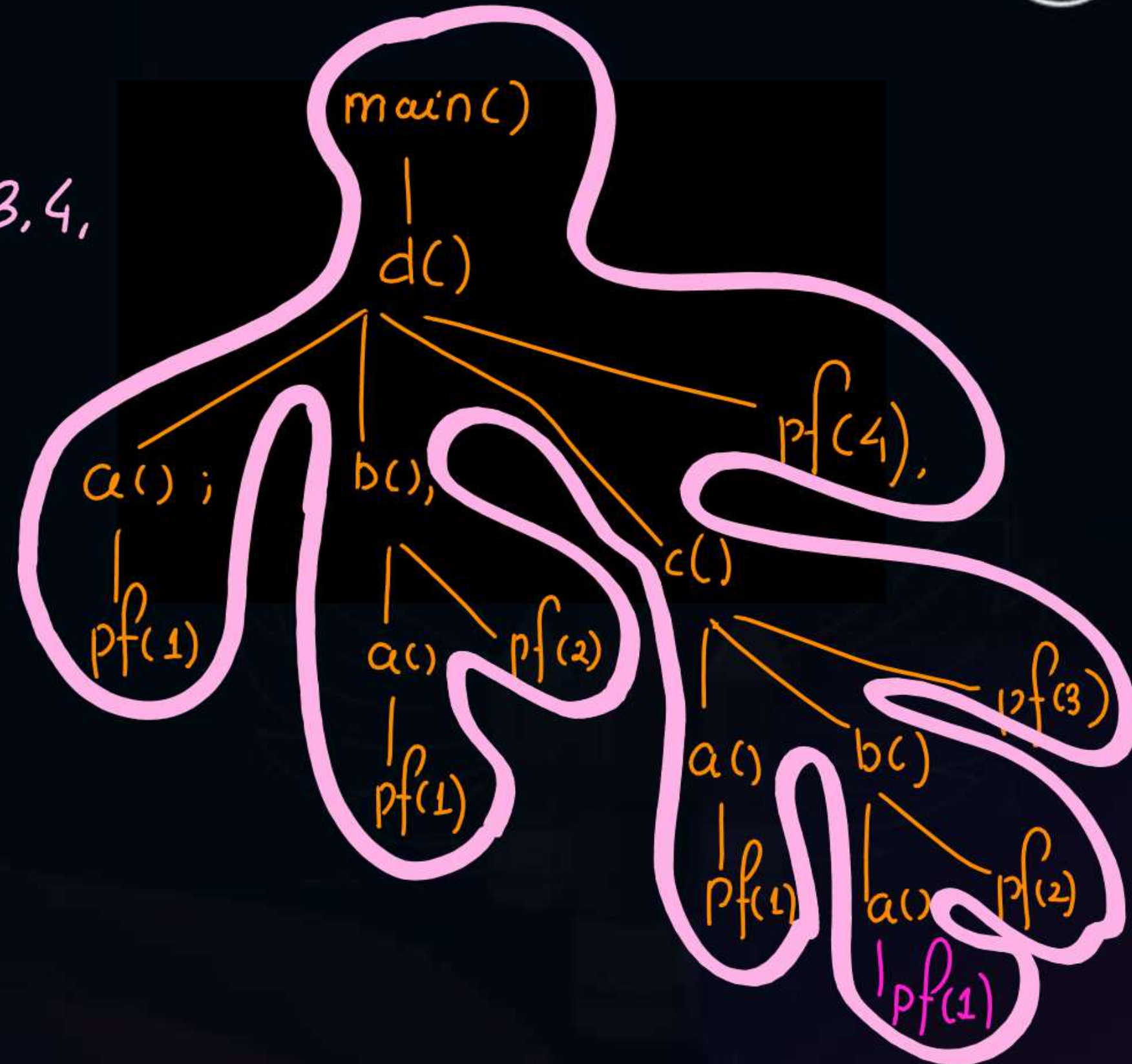
Question



#Q Consider the following program

```
#include <stdio.h> 1, 1, 2 1 1 2, 3, 4,  
void a(){printf("1");}  
void b(){a();printf("2");}  
void c(){a();b();printf("3");}  
void d(){a();b();c();printf("4");}  
  
int main(){  
    d();  
}
```

output - trace of call
tracing of tree





Question



#Q

Consider the following C program:

```
#include<stdio.h>
```

```
int f1(void);
```

```
int f2(void);
```

```
int f3(void);
```

```
int x=10;
```

```
int main()
```

```
{
```

```
    int x=1;
```

```
    x += f1() + f2() + f3() + f2();
```

```
    printf("%d", x);
```

```
    return 0;
```

```
}
```

✓ 100 update

for (i=0; i<=10; i++)

++x

```
int f1() { int x = 25; x++; return x; }
```

```
int f2() { int x = 50; x++; return x; }
```

```
int f3() { x *= 10; return x; }
```

The output of the program is _____.

$$1 + 26 + 51 + 100 + 51 = \boxed{229}$$

Local vs. global
 x_L x_g

Local will be preferred.



Question

```
#include <stdio.h>
int a=10;
int fun(int i) {
    while (i<20){
        i+=2;
        if(i%2) continue;
        i+=3;
    }
    a += i;
    return a+i ;
}

int main(){
    printf("%d", fun(fun(10)));
    return 0;
```

$a : a + i = 10 + 21 = 31$

$31 + 21 = 52$

$10 < 20$

$i = 12$

if(odd) continue

$i += 3$ $i = 15$

$17 < 20$

19

$15 < 20$

$i = 17$

$19 < 20$

21



Question

```
#include <stdio.h>
int a=10;
int fun(int i) {
    while (i<20){
        i+=2;
        if(i%2) continue;
        i+=3;
    }
    a +=i;
    return a+i ;
}

int main(){
    printf("%d", fun(fun(10)));
    return 0;
```

$$52 < 20$$

$$a = 31 + 52 = 83$$

$$83 + 52 = 135 \checkmark$$

$$= \text{fun}(52)$$

+ me/Abhishekshoampw

THANK - YOU

